

MACHINE REPRESENTATION OF PROGRAMS

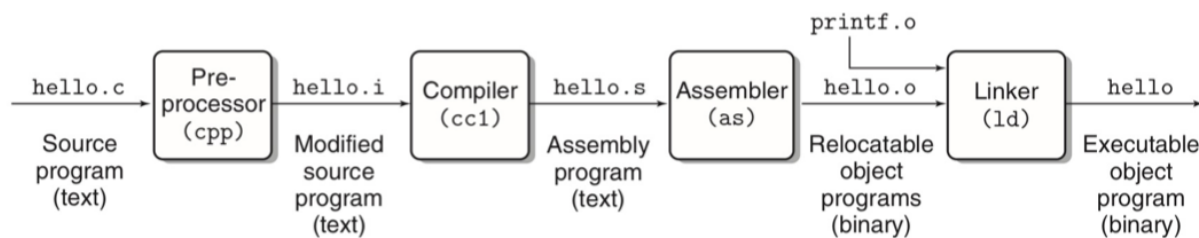
Computers execute machine code

↳ sequence of bytes encoding low level operations

A compiler generates machine code by-

- Rules of the programming language
- Instruction set of the target machine
- Conventions of the Operating System

→ The Compilation Process



Compiler Optimizations can rearrange execution order, eliminate unwanted computations, replace slow operations and change recursive computation to iterative ones.

Moore's Law - The number of transistors per chip will double every year.

↳ Moore's Law has sadly ended. I believe Physics caught up.

The format and behavior of machine level programs is defined by the instruction set architecture. Most ISAs describe the behavior of the program as if each instruction is executed in sequence, with one instruction completing before the next one begins.

High level languages provide a model in which objects of different data types can be declared and allocated in memory, machine code views the memory as simply a large byte-addressable array.

Program memory is addressed using virtual addresses. The OS manages the virtual address space, translating virtual addresses into the physical addresses.

To view the assembly code generated by the C compiler-

> gcc -Og -S hello.c

-Og corresponds to the level of compiler optimization

Disassemblers allow us to inspect contents of the machine level code. They generate a format similar to assembly code from machine code.

x86-64 instructions can range in length from 1 to 15 bytes. Commonly used instructions and those with fewer operands require a smaller number of bytes.

Most assembly code instructions generated by GCC have a single character suffix denoting the size of the operand. Floating point code involves an entirely different set of instructions and registers.

An x86-64 CPU contains a set of 16 general purpose registers storing 64-bit values. These registers are used to store integer data as well as pointers -

%rax → Return Value

%rbx → Caller Saved

%rcx → 4th Argument

%rdx → 3rd Argument

%rsi → 2nd Argument

%rdi → 1st Argument

%rbp → Caller Saved

%rsp → Stack Pointer

%r8 → 5th Argument

%r9 → 6th Argument

%r10 → Caller Saved

%r11 → Caller Saved

%r12 → Caller Saved

%r13 → Caller Saved

%r14 → Caller Saved

%r15 → Caller Saved

→ What happens if we have more than 6 arguments?

• The excess arguments are placed on the stack. Note that accessing the stack is slower than registers.

Assembly Instructions - Instruction Operand [Operand]

↳ Operands are generally source & destination locations

constants, registers, memory ←

↳ registers & memory

→ Data Movement Instructions -

Mov S, D → This instruction copies data from a source location to a destination location without any transformation.

↳ x86 restricts both operands from referencing memory locations. This is because CPU buses are designed to only make one memory reference at a time. Copying a value from one memory location to another requires two instructions: the first to load the source value into a register and the second to write this register to the destination.

MOVZ S, R → Copies a smaller value to a larger destination.

↳ Fills the remaining bytes of the destination with zeroes.

MOVS S, R → Copies a smaller value to a larger destination.

↳ Fills the remaining bytes of the destination with sign extension (replicating copies of the MSB of the source operand).

NOTE: For most Mov instructions, only the specific register bytes mentioned by the operand are updated. However, when we have a 32 bit value to a 64 bit destination, the upper 32 bits are zeroed out.

NOTE: Pointers in C are simply addresses. Dereferencing a pointer involves copying that pointer into a register, and then using this register in a memory access.

NOTE: Local variables are stored in registers as opposed to memory. Register access is faster than memory access.

→ The Stack -

A data structure where data is added in a last in first out manner. It plays a very important role in procedure calls.

↳ In x86, the stack grows downwards such that the top element always has the lowest address of all stack elements. The stack pointer `%rsp` holds the address of the top element.

Pushing
`subq $8, %rsp`
`movq %rbp, (%rsp)` } `pushq %rbp`

Popping
`movq (%rsp), %rax`
`addq $8, %rsp` } `popq %rax`

Since the stack is contained in the same memory as the program code and other forms of program data, programs can access random positions within the stack using standard memory addressing methods.

→ Arithmetic and Logical Operations -

`lshl S,D`: $D \leftarrow S \ll S$

`inc D`: $D \leftarrow D + 1$

`dec D`: $D \leftarrow D - 1$

`neg D`: $D \leftarrow -D$

`not D`: $D \leftarrow \sim D$

`add S,D`: $D \leftarrow S + D$

`sub S,D`: $D \leftarrow D - S$

`imul S,D`: $D \leftarrow D * S$

`xor S,D`: $D \leftarrow S \oplus D$

`or S,D`: $D \leftarrow D \vee S$

`and S,D`: $D \leftarrow D \wedge S$

`sll R,D`: $D \leftarrow D \ll R$

`sar R,D`: $D \leftarrow D \gg_A R \rightarrow$ Arithmetic

`shr R,D`: $D \leftarrow D \gg_L R \rightarrow$ Logical

As with the `mov` instructions, operands to binary instructions cannot both be memory addresses owing to the fact that CPU buses cannot have two memory accesses simultaneously.

→ Control Instructions -

Machine code provides two basic low level mechanisms for implementing conditional behavior - it tests data values and either alters the data or the control flow based on these tests. Data dependent control flow is the more general approach for implementing conditional behaviors.

Generally instructions in both C and machine code are instructed sequentially, i.e., in the order in which they appear in the program.

↳ The execution order can be altered with the jump instruction. This indicates that control should pass to some other part of the program.

The CPU maintains a set of single bit condition code registers describing attributes of the most recent arithmetic or logical operation.

- CF - Carry Flag
- ZF - Zero Flag
- SF - Sign Flag
- OF - Overflow Flag

→ The CMP behaves as the SUB instruction

→ The TEST behaves as the AND instruction

↳ Both these instructions do not update the destination operands. They only update the condition codes.

Condition codes are not accessed directly. Instead they are used indirectly-

- SET - Set a single byte to 0 or 1 based on some condition.
- JMP - Conditionally jump to some other part of the program.
- CMov - Conditionally transfer data

→ Jump Instructions -

Under normal execution, instructions follow each other in the order they are listed. A jump instruction can cause the execution to switch to a new position in the program. Jump destinations are generally indicated by a label.

While generating object code file, the assembler determines the address of all labeled instructions and encodes the jump targets as part of the instruction.

Jump instructions can be either conditional or unconditional. Jump targets can either be -

↳ Conditional jumps can only be direct.

1. Direct - the target is encoded as part of the instruction.

2. Indirect - the target is either a register or a memory location.

↳ Direct jumps are written by giving a label as a target → `jmp .L1`

↳ Indirect jumps are written using a * followed by a location → `jmp *(%rax)`

In assembly code, jump targets are written using symbolic labels. The assembler and later the linker generate the proper encodings of the jump targets.

Jump encodings are generally PC relative. PC → Program Counter

↳ They encode difference between the address of the target instruction and the instruction immediately following the jump.

↳ The value of the PC is the address of the instruction following the jump. This is done for historical reasons where the processor would update the PC as the first step in executing an instruction

↳ By using a PC relative encoding of jump targets, the instructions can be compactly encoded, and the object code can be shifted to different positions in memory without alteration.

Processors achieve high performance through pipelining, where an instruction is processed via a series of stages, each performing one small portion of the operation. This approach achieves high performance by overlapping the steps of successive instructions. To do this requires knowledge of the order of instruction execution ahead of time.

↳ This cannot happen reliably when a processor encounters a conditional jump. Processors apply sophisticated branch prediction algorithms to help.

If branch prediction fails, the program incurs significant penalties. This can be avoided by using conditional moves as opposed to conditional jumps. The processor simply reads the source value, checks the condition code, and then either updates the destination or keeps it the same.

↳ A conditional move in C is the ternary operator -
 $v = \text{test-expr} ? \text{then-expr} : \text{else-expr}$

Conditional moves are efficient as they evaluate the result hands before the conditional assign. However, if either of the hands is computationally intensive, conditional moves would perform worst. It is also the reason why we can't use them for pointer dereferencing -

$p ? *p : 0 \rightarrow *p$ and 0 are evaluated first before checking if p is a non NULL pointer.

→ Loops -

C has many looping constructs - do while, while & for. There is no machine code equivalent. ISA's use a combination of conditional tests and jumps to implement loops.

